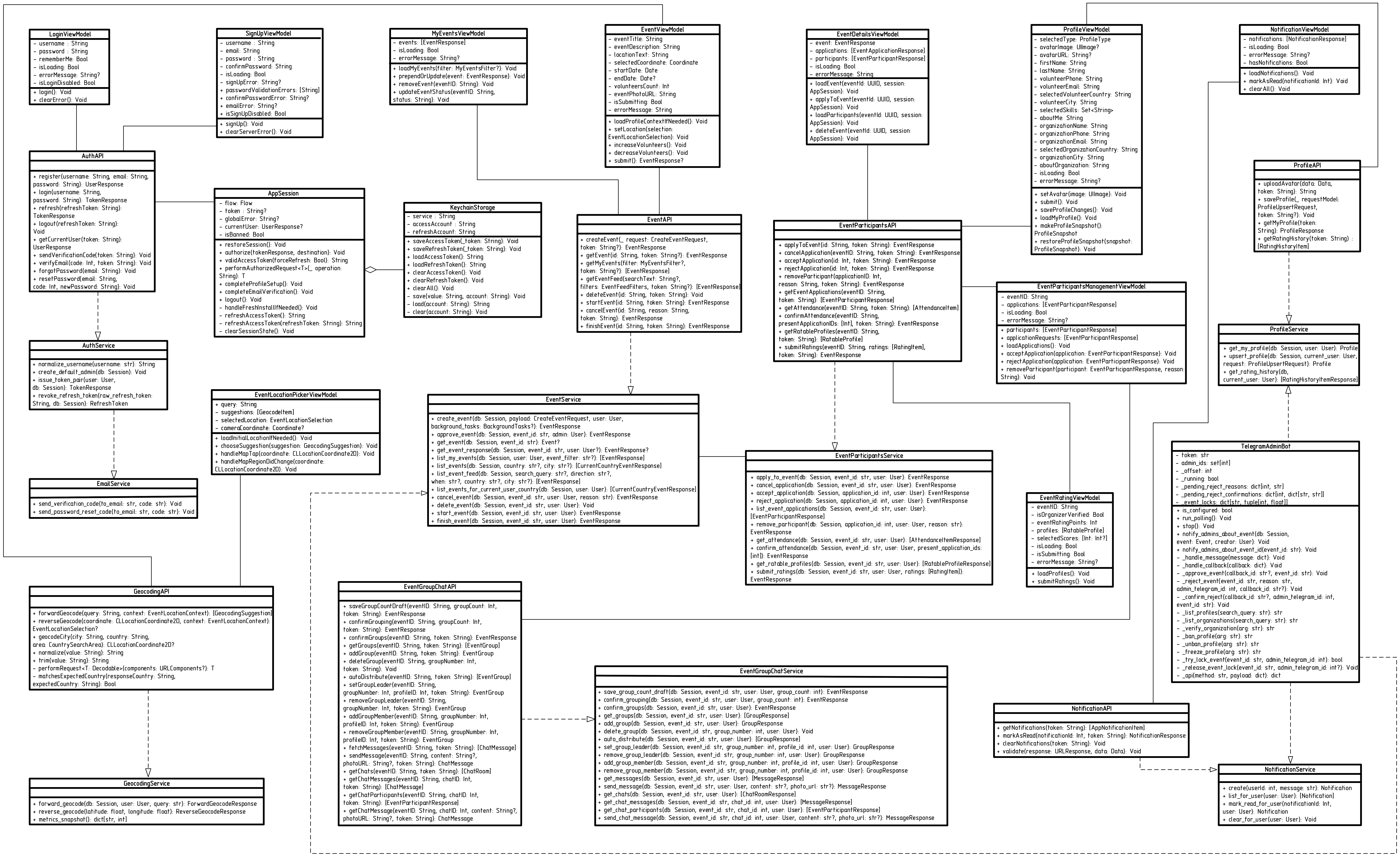
ГУИР.400201.311 РР.1


LoginViewModel
- username : String
- password : String
- rememberMe: Bool
- isLoading: Bool
- errorMessage: String?
- isLoginDisabled: Bool
+ login(): Void
+ clearError(): Void
SignUpViewModel
- username : String
- email: String
- password : String
- confirmPassword: String
- isLoading: Bool
- signUpError: String?
+ passwordValidationErrors: [String]
+ confirmPasswordError: String?
+ emailError: String?
+ isSignUpDisabled: Bool
+ signUp(): Void
+ clearServerError(): Void
MyEventsViewModel
- events: [EventResponse]
- isLoading: Bool
- errorMessage: String?
+ loadMyEvents(filter: MyEventsFilter?): Void
+ prependOrUpdate(event: EventResponse): Void
+ removeEvent(eventID: String): Void
+ updateEventStatus(eventID: String,
status: String): Void
EventViewModel
- eventTitle: String
- eventDescription: String
- locationText: String
- selectedCoordinate: Coordinate
- startDate: Date
- endDate: Date?
- volunteersCount: Int
- eventPhotoURL: String
- isSubmitting: Bool
- errorMessage: String
+ loadProfileContextIfNeeded(): Void
+ setLocation(selection:
EventLocationSelection): Void
+ increaseVolunteers(): Void
+ decreaseVolunteers(): Void
+ submit(): EventResponse?
EventDetailsViewModel
- event: EventResponse
- applications: [EventApplicationResponse]
- participants: [EventParticipantResponse]
- isLoading: Bool
- errorMessage: String
+ loadEvent(eventId: UUID, session:
AppSession): Void
+ applyToEvent(eventId: UUID, session:
AppSession): Void
+ loadParticipants(eventId: UUID, session:
AppSession): Void
+ deleteEvent(eventId: UUID, session:
AppSession): Void
ProfileViewModel
- selectedType: ProfileType
- avatarImage: UIImage?
- avatarURL: String?
- firstName: String
- lastName: String
- volunteerPhone: String
- volunteerEmail: String
- selectedVolunteerCountry: String
- volunteerCity: String
- selectedSkills: Set<String>
- aboutMe: String
- organizationName: String
- organizationPhone: String
- organizationEmail: String
- selectedOrganizationCountry: String
- organizationCity: String
- aboutOrganization: String
- isLoading: Bool
- errorMessage: String?
+ setAvatar(image: UIImage): Void
+ submit(): Void
+ saveProfileChanges(): Void
+ loadMyProfile(): Void
+ makeProfileSnapshot():
ProfileSnapshot
+ restoreProfileSnapshot(snapshot:
ProfileSnapshot): Void
NotificationViewModel
- notifications: [NotificationResponse]
- isLoading: Bool
- errorMessage: String?
- hasNotifications: Bool
+ loadNotifications(): Void
+ markAsRead(notificationId: Int): Void
+ clearAll(): Void
AuthAPI
+ register(username: String, email: String,
password: String): UserResponse
+ login(username: String,
password: String): TokenResponse
+ refresh(refreshToken: String):
TokenResponse
+ logout(refreshToken: String):
Void
+ getCurrentUser(token: String):
UserResponse
+ sendVerificationCode(token: String): Void
+ verifyEmail(code: Int, token: String): Void
+ forgotPassword(email: String): Void
+ resetPassword(email: String,
code: Int, newPassword: String): Void
AppSession
- flow: Flow
- token : String?
- globalError: String?
- currentUser: UserResponse?
- isBanned: Bool
+ restoreSession(): Void
+ authorize(tokenResponse, destination): Void
+ validAccessToken(forceRefresh: Bool): String
+ performAuthorizedRequest<T>(_ operation:
String): T
+ completeProfileSetup(): Void
+ completeEmailVerification(): Void
+ logout(): Void
- handleFreshInstallIfNeeded(): Void
- refreshAccessToken(): String
- refreshAccessToken(refreshToken: String): String
- clearSessionState(): Void
KeychainStorage
- service : String
- accessAccount : String
- refreshAccount: String
+ saveAccessToken(_token: String): Void
+ saveRefreshToken(_token: String): Void
+ loadAccessToken(): String
+ loadRefreshToken(): String
+ clearAccessToken(): Void
+ clearRefreshToken(): Void
+ clearAll(): Void
- save(value: String, account: String): Void
- load(account: String): String
- clear(account: String): Void
EventAPI
+ createEvent(_ request: CreateEventRequest,
token: String?): EventResponse
+ getEvent(id: String, token: String?): EventResponse
+ getMyEvents(filter: MyEventsFilter?,
token: String?): [EventResponse]
+ getEventFeed(searchText: String?,
filters: EventFeedFilters, token: String?): [EventResponse]
+ deleteEvent(id: String, token: String): Void
+ startEvent(id: String, token: String): EventResponse
+ cancelEvent(id: String, reason: String,
token: String): EventResponse
+ finishEvent(id: String, token: String): EventResponse
EventParticipantsAPI
+ applyToEvent(id: String, token: String): EventResponse
+ cancelApplication(eventID: String, token: String): EventResponse
+ acceptApplication(id: Int, token: String): EventResponse
+ rejectApplication(id: Int, token: String): EventResponse
+ removeParticipant(applicationID: Int,
reason: String, token: String): EventResponse
+ getEventApplications(eventID: String,
token: String): [EventParticipantResponse]
+ getAttendance(eventID: String, token: String): [AttendanceItem]
+ confirmAttendance(eventID: String,
presentApplicationIDs: [Int], token: String): EventResponse
+ getRatableProfiles(eventID: String,
token: String): [RatableProfile]
+ submitRatings(eventID: String, ratings: [RatingItem],
token: String): EventResponse
ProfileAPI
+ uploadAvatar(data: Data,
token: String): String
+ saveProfile(_ requestModel:
ProfileUpsertRequest,
token: String?): Void
+ getMyProfile(token:
String): ProfileResponse
+ getRatingHistory(token: String) :
[RatingHistoryItem]
EventParticipantsManagementViewModel
- eventID: String
- applications: [EventParticipantResponse]
- isLoading: Bool
- errorMessage: String?
+ participants: [EventParticipantResponse]
+ applicationRequests: [EventParticipantResponse]
+ loadApplications(): Void
+ acceptApplication(application: EventParticipantResponse): Void
+ rejectApplication(application: EventParticipantResponse): Void
+ removeParticipant(participant: EventParticipantResponse, reason:
String): Void
ProfileService
+ get_my_profile(db: Session, user: User): Profile
+ upsert_profile(db: Session, current_user: User,
request: ProfileUpsertRequest): Profile
+ get_rating_history(db,
current_user: User): [RatingHistoryItemResponse]
AuthService
+ normalize_username(username: str): String
+ create_default_admin(db: Session): Void
+ issue_token_pair(user: User,
db: Session): TokenResponse
+ revoke_refresh_token(raw_refresh_token:
String, db: Session): RefreshToken
EventLocationPickerViewModel
+ query: String
- suggestions: [GeocodeItem]
- selectedLocation: EventLocationSelection
- cameraCoordinate: Coordinate?
+ loadInitialLocationIfNeeded(): Void
+ chooseSuggestion(suggestion: GeocodingSuggestion): Void
+ handleMapTap(coordinate: CLLocationCoordinate2D): Void
+ handleMapRegionDidChange(coordinate:
CLLocationCoordinate2D): Void
EventService
+ create_event(db: Session, payload: CreateEventRequest, user: User,
background_tasks: BackgroundTasks?): EventResponse
+ approve_event(db: Session, event_id: str, admin: User): EventResponse
+ get_event(db: Session, event_id: str): Event?
+ get_event_response(db: Session, event_id: str, user: User?): EventResponse?
+ list_my_events(db: Session, user: User, event_filter: str?): [EventResponse]
+ list_events(db: Session, country: str?, city: str?): [CurrentCountryEventResponse]
+ list_event_feed(db: Session, search_query: str?, direction: str?,
when: str?, country: str?, city: str?): [EventResponse]
+ list_events_for_current_user_country(db: Session, user: User): [CurrentCountryEventResponse]
+ cancel_event(db: Session, event_id: str, user: User, reason: str): EventResponse
+ delete_event(db: Session, event_id: str, user: User): Void
+ start_event(db: Session, event_id: str, user: User): EventResponse
+ finish_event(db: Session, event_id: str, user: User): EventResponse
EventParticipantsService
+ apply_to_event(db: Session, event_id: str, user: User): EventResponse
+ cancel_application(db: Session, event_id: str, user: User): EventResponse
+ accept_application(db: Session, application_id: int, user: User): EventResponse
+ reject_application(db: Session, application_id: int, user: User): EventResponse
+ list_event_applications(db: Session, event_id: str, user: User):
[EventParticipantResponse]
+ remove_participant(db: Session, application_id: int, user: User, reason: str):
EventResponse
+ get_attendance(db: Session, event_id: str, user: User): [AttendanceItemResponse]
+ confirm_attendance(db: Session, event_id: str, user: User, present_application_ids:
[int]): EventResponse
+ get_ratable_profiles(db: Session, event_id: str, user: User): [RatableProfileResponse]
+ submit_ratings(db: Session, event_id: str, user: User, ratings: [RatingItem]):
EventResponse
TelegramAdminBot
- token: str
- admin_ids: set[int]
- _offset: int
- _running: bool
- _pending_reject_reasons: dict[int, str]
- _pending_reject_confirmations: dict[int, dict[str, str]]
- _event_locks: dict[str, tuple[int, float]]
+ is_configured: bool
+ run_polling(): Void
+ stop(): Void
+ notify_admins_about_event(db: Session,
event: Event, creator: User): Void
+ notify_admins_about_event_id(event_id: str): Void
- _handle_message(message: dict): Void
- _handle_callback(callback: dict): Void
- _approve_event(callback_id: str?, event_id: str): Void
- _reject_event(event_id: str, reason: str,
admin_telegram_id: int, callback_id: str?): Void
- _confirm_reject(callback_id: str?, admin_telegram_id: int,
event_id: str): Void
- _list_profiles(search_query: str): str
- _list_organizations(search_query: str): str
- _verify_organization(arg: str): str
- _ban_profile(arg: str): str
- _unban_profile(arg: str): str
- _freeze_profile(arg: str): str
- _try_lock_event(event_id: str, admin_telegram_id: int): bool
- _release_event_lock(event_id: str, admin_telegram_id: int?): Void
- _api(method: str, payload: dict): dict
EmailService
+ send_verification_code(to_email: str, code: str): Void
+ send_password_reset_code(to_email: str, code: str): Void
EventRatingViewModel
- eventID: String
- isOrganizerVerified: Bool
- eventRatingPoints: Int
- profiles: [RatableProfile]
- selectedScores: [Int: Int?]
- isLoading: Bool
- isSubmitting: Bool
- errorMessage: String?
+ loadProfiles(): Void
+ submitRatings(): Void
GeocodingAPI
+ forwardGeocode(query: String, context: EventLocationContext): [GeocodingSuggestion]
+ reverseGeocode(coordinate: CLLocationCoordinate2D, context: EventLocationContext):
EventLocationSelection?
+ geocodeCity(city: String, country: String,
area: CountrySearchArea): CLLocationCoordinate2D?
+ normalize(value: String): String
+ trim(value: String): String
- performRequest<T: Decodable>(components: URLComponents?): T
- matchesExpectedCountry(responseCountry: String,
expectedCountry: String): Bool
EventGroupChatAPI
+ saveGroupCountDraft(eventID: String, groupCount: Int,
token: String): EventResponse
+ confirmGrouping(eventID: String, groupCount: Int,
token: String): EventResponse
+ confirmGroups(eventID: String, token: String): EventResponse
+ getGroups(eventID: String, token: String): [EventGroup]
+ addGroup(eventID: String, token: String): EventGroup
+ deleteGroup(eventID: String, groupNumber: Int,
token: String): Void
+ autoDistribute(eventID: String, token: String): [EventGroup]
+ setGroupLeader(eventID: String,
groupNumber: Int, profileID: Int, token: String): EventGroup
+ removeGroupLeader(eventID: String,
groupNumber: Int, token: String): EventGroup
+ addGroupMember(eventID: String, groupNumber: Int,
profileID: Int, token: String): EventGroup
+ removeGroupMember(eventID: String, groupNumber: Int,
profileID: Int, token: String): EventGroup
+ fetchMessages(eventID: String, token: String): [ChatMessage]
+ sendMessage(eventID: String, content: String?,
photoURL: String?, token: String): ChatMessage
+ getChats(eventID: String, token: String): [ChatRoom]
+ getChatMessages(eventID: String, chatID: Int,
token: String): [ChatMessage]
+ getChatParticipants(eventID: String, chatID: Int,
token: String): [EventParticipantResponse]
+ getChatMessage(eventID: String, chatID: Int, content: String?,
photoURL: String?, token: String): ChatMessage
EventGroupChatService
+ save_group_count_draft(db: Session, event_id: str, user: User, group_count: int): EventResponse
+ confirm_grouping(db: Session, event_id: str, user: User, group_count: int): EventResponse
+ confirm_groups(db: Session, event_id: str, user: User): EventResponse
+ get_groups(db: Session, event_id: str, user: User): [GroupResponse]
+ add_group(db: Session, event_id: str, user: User): GroupResponse
+ delete_group(db: Session, event_id: str, group_number: int, user: User): Void
+ auto_distribute(db: Session, event_id: str, user: User): [GroupResponse]
+ set_group_leader(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse
+ remove_group_leader(db: Session, event_id: str, group_number: int, user: User): GroupResponse
+ add_group_member(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse
+ remove_group_member(db: Session, event_id: str, group_number: int, profile_id: int, user: User): GroupResponse
+ get_messages(db: Session, event_id: str, user: User): [MessageResponse]
+ send_message(db: Session, event_id: str, user: User, content: str?, photo_url: str?): MessageResponse
+ get_chats(db: Session, event_id: str, user: User): [ChatRoomResponse]
+ get_chat_messages(db: Session, event_id: str, chat_id: int, user: User): [MessageResponse]
+ get_chat_participants(db: Session, event_id: str, chat_id: int, user: User): [EventParticipantResponse]
+ send_chat_message(db: Session, event_id: str, chat_id: int, user: User, content: str?, photo_url: str?): MessageResponse
GeocodingService
+ forward_geocode(db: Session, user: User, query: str): ForwardGeocodeResponse
+ reverse_geocode(latitude: float, longitude: float): ReverseGeocodeResponse
+ metrics_snapshot(): dict[str, int]
NotificationAPI
+ getNotifications(token: String): [AppNotificationItem]
+ markAsRead(notificationId: Int, token: String): NotificationResponse
+ clearNotifications(token: String): Void
+ validate(response: URLResponse, data: Data): Void
NotificationService
+ create(userId: int, message: str): Notification
+ list_for_user(user: User): [Notification]
+ mark_read_for_user(notificationId: Int,
user: User): Notification
+ clear_for_user(user: User): Void

ГУИР.400201.311 РР.1

Программное средство для организации и управления волонтёрской деятельностью. Диаграмма классов

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.		Прокопюк		
Пров.		Селезнев		
Т. Контр.		Селезнев		
Реценз.				
Н. Контр.		Ковшер		
Утв.		Никульшин		

Лит.	Масса	Масштаб
Т		

Лист	Листов 1

ЭВМ, гр. 150541